Employee Management Page (App-task:6)

//code : employee_page.dart

```
import 'package:flutter/material.dart';
import 'package:cloud_firestore/cloud_firestore.dart';

// Task 1 — Employee Model
class Employee {
 final String? id;                                    //can be null
 final String name;
 final int age;
 final String department;
 final double salary;
 final String email;
 Employee({                                           //Constructors
  this.id,
  required this.name,
  required this.age,
  required this.department,                           // Required field must be provided
  required this.salary,
  required this.email,
 });
 Map<String, dynamic> toMap() {                       // converts to map which firestore understands
  return {
   'name': name,
   'age': age,
   'department': department,
   'salary': salary,
   'email': email,
  };
 }
 factory Employee.fromFirestore(DocumentSnapshot doc) {      // firestore return ->dart object
```

FLUTTER MINI ASSIGNMENT

```
  final data = doc.data() as Map<String, dynamic>;
  return Employee(
    id: doc.id,
    name: data['name'],
    age: data['age'],
    department: data['department'],
    salary: (data['salary'] as num).toDouble(),
    email: data['email'],
  );
 }
}
```

//Task 2 — EmployeeService

```
class EmployeeService {
 final _col = FirebaseFirestore.instance.collection('employees');          //employees collection in Firestore
 Stream<List<Employee>> getEmployees() => _col                                  //getEmployee()
    .orderBy('name')                                                            //Sorting
    .snapshots()                                                                // Real time read
    .map((s) => s.docs.map((d) => Employee.fromFirestore(d)).toList());
 Future<void> addEmployee(Employee e) => _col.add(e.toMap());                   // Add new employee
 Future<void> updateEmployee(Employee e) => _col.doc(e.id).update(e.toMap());      // Update employee
 Future<void> deleteEmployee(String id) => _col.doc(id).delete();                  //Delete Employee
 Stream<List<Employee>> getEmployeesByDepartment(String dept) => _col
    .where('department', isEqualTo: dept)                                       //Filter by dept
    .orderBy('name')
    .snapshots()
    .map((s) => s.docs.map((d) => Employee.fromFirestore(d)).toList());
}
```

//Task 3 — CRUD UI

```
class EmployeesPage extends StatefulWidget {                                    //main screen
 const EmployeesPage({super.key});
 @override
```

FLUTTER MINI ASSIGNMENT

```
 State<EmployeesPage> createState() => _EmployeesPageState();
}
class _EmployeesPageState extends State<EmployeesPage> {
 final _service = EmployeeService();
 String _searchQuery = '';                                                    //Search Querry
 bool _isSearching = false;
 String _selectedDept = "All";                                                //Filter Querry
 final List<String> _departments = ["All", "HR", "IT", "Finance", "Marketing"];   //Filter chips
 Stream<List<Employee>> get _activeStream {
  if (_selectedDept == "All") return _service.getEmployees();                 //Apply filter
  return _service.getEmployeesByDepartment(_selectedDept);
 }
 List<Employee> _applySearch(List<Employee> employees) {      //Search logic through name and email
  if (_searchQuery.isEmpty) return employees;
  return employees
    .where(
     (e) =>
       e.name.toLowerCase().contains(_searchQuery.toLowerCase()) ||          //case sensitive
       e.email.toLowerCase().contains(_searchQuery.toLowerCase()),
    )
    .toList();
 }
 void _openForm({Employee? employee}) {
  showModalBottomSheet(
   context: context,
   isScrollControlled: true,
   builder: (_) => _EmployeeForm(
    employee: employee,
    onSave: (e) async {
     if (e.id == null) {
      await _service.addEmployee(e);
     } else {
      await _service.updateEmployee(e);
```

```
      }
     },
    ),
  );
}
void _confirmDelete(Employee e) {                                    //Delete Confirmation Dialog
  showDialog(
    context: context,
    builder: (_) => AlertDialog(
      title: Text("Delete Employee"),
      content: Text("Delete ${e.name}?"),
      actions: [
        TextButton(
          onPressed: () => Navigator.pop(context),
          child: Text("Cancel"),
        ),
        FilledButton(
          style: FilledButton.styleFrom(backgroundColor: Colors.red),
          onPressed: () async {
            await _service.deleteEmployee(e.id!);
            Navigator.pop(context);
          },
          child: Text("Delete"),
        ),
      ],
    ),
  );
}
```

//Task 4 — Search Feature

```
@override
Widget build(BuildContext context) {
  return Scaffold(
```

FLUTTER MINI ASSIGNMENT

```
appBar: AppBar(
  title: _isSearching
      ? TextField(
          autofocus: true,
          decoration: InputDecoration(
            hintText: "Search by name or email",
            border: InputBorder.none,
          ),
          onChanged: (val) => setState(() => _searchQuery = val),
        )
      : Text("Employees"),
  actions: [
    IconButton(                                                    //Toggle Search
      icon: Icon(_isSearching ? Icons.close : Icons.search),
      onPressed: () {
        setState(() {
          _isSearching = !_isSearching;
          _searchQuery = '';
        });
      },
    ),
  ],
),
floatingActionButton: FloatingActionButton.extended(             //Employee button opens the form
  onPressed: () => _openForm(),
  icon: Icon(Icons.person_add),
  label: Text("Add Employee"),
),
body: Column(
  children: [
    Container(
      padding: EdgeInsets.all(10),
      child: SingleChildScrollView(
```

```
    scrollDirection: Axis.horizontal,
    child: Row(
      children: _departments.map((dept) {
        final selected = _selectedDept == dept;
        return Padding(
          padding: EdgeInsets.only(right: 8),

          // Task 5 — Filter by Department
          child: ChoiceChip(
            label: Text(dept),
            selected: selected,
            onSelected: (_) => setState(() => _selectedDept = dept),
          ),
        );
      }).toList(),
    ),
  ),
),
Expanded(
  child: StreamBuilder<List<Employee>>(                    //Listens to Firestore live updates
    stream: _activeStream,
    builder: (context, snapshot) {
      if (snapshot.connectionState == ConnectionState.waiting) {
        return Center(child: CircularProgressIndicator());
      }

      // Bonus Task
      final employees = _applySearch(snapshot.data ?? []);
      int total = employees.length;
      double avgSalary = 0;
      if (employees.isNotEmpty) {                                          //Calculations
        avgSalary =
            employees.map((e) => e.salary).reduce((a, b) => a + b) /
```

```
    employees.length;
}
if (employees.isEmpty) {
  return Center(child: Text("No employees found"));                    //No Results
}
return Column(
  children: [
    Padding(
      padding: EdgeInsets.symmetric(horizontal: 16),
      child: Row(
        children: [
          Text(
            "Showing $total Employees",
            style: TextStyle(
              fontWeight: FontWeight.bold,
              color: Colors.blue,
            ),
          ),
          if (_selectedDept != "All") ...[
            SizedBox(width: 8),
            Container(
              padding: EdgeInsets.symmetric(
                horizontal: 8,
                vertical: 2,
              ),
              decoration: BoxDecoration(
                color: Colors.purple,
                borderRadius: BorderRadius.circular(20),
              ),
              child: Text(
                _selectedDept,
                style: TextStyle(
                  fontSize: 11,
```

```
            color: Colors.white,
          ),
        ),
      ),
    ],
  ],
 ),
),
Expanded(
 child: ListView.builder(
  padding: EdgeInsets.all(12),
  itemCount: employees.length,
  itemBuilder: (_, i) {
   final e = employees[i];
   return _EmployeeCard(
    employee: e,
    onEdit: () => _openForm(employee: e),
    onDelete: () => _confirmDelete(e),
   );
  },
 ),
),
Container(
 padding: EdgeInsets.all(12),
 color: Colors.grey.shade200,
 child: Row(
  mainAxisAlignment: MainAxisAlignment.spaceBetween,
  children: [
   Text(
    "Total Employees: $total",
    style: TextStyle(fontWeight: FontWeight.bold),
   ),
   Text(
```

```
                "Avg Salary: ₹${avgSalary.toStringAsFixed(0)}",
                style: TextStyle(fontWeight: FontWeight.bold),
              ),
            ],
          ),
        ),
      ],
    );
  },
),
),
],
),
);
}
}
class _EmployeeCard extends StatelessWidget {                                    //Displays employees
 final Employee employee;
 final VoidCallback onEdit;
 final VoidCallback onDelete;
 const _EmployeeCard({
  required this.employee,
  required this.onEdit,
  required this.onDelete,
 });
 @override
 Widget build(BuildContext context) {
  return Card(
   child: ListTile(
    leading: CircleAvatar(child: Text(employee.name[0].toUpperCase())),
//First letter of name as circle like logo
    title: Text(employee.name),
    subtitle: Column(
```

```
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Text(employee.email),
          Wrap(
            spacing: 6,
            runSpacing: 4,
            children: [
              _Chip(label: "Age ${employee.age}"),                    //small tag like chips
              SizedBox(width: 6),
              _Chip(label: employee.department),
              SizedBox(width: 6),
              _Chip(label: "₹${employee.salary}"),
            ],
          ),
        ],
      ),
      trailing: Row(
        mainAxisSize: MainAxisSize.min,
        children: [
          IconButton(icon: Icon(Icons.edit), onPressed: onEdit),          //Edit Button
          IconButton(                                                     //Delete Button
            icon: Icon(Icons.delete),
            color: Colors.red,
            onPressed: onDelete,
          ),
        ],
      ),
    ),
  );
}
}
class _Chip extends StatelessWidget {
  final String label;
```

```
 const _Chip({required this.label});
 @override
 Widget build(BuildContext context) {
  return Container(
    padding: EdgeInsets.symmetric(horizontal: 8, vertical: 2),
    decoration: BoxDecoration(
      color: Colors.blueGrey,
      borderRadius: BorderRadius.circular(20),
    ),
    child: Text(label, style: TextStyle(color: Colors.white, fontSize: 11)),
  );
 }
}

class _EmployeeForm extends StatefulWidget {
//Bottomsheet form appear when adding student button clicked
 final Employee? employee;
 final Future<void> Function(Employee) onSave;
 const _EmployeeForm({this.employee, required this.onSave});
 @override
 State<_EmployeeForm> createState() => _EmployeeFormState();
}
class _EmployeeFormState extends State<_EmployeeForm> {
 final _formKey = GlobalKey<FormState>();
 late TextEditingController _name;                                    //latek-> controllers
 late TextEditingController _age;
 late TextEditingController _department;
 late TextEditingController _salary;
 late TextEditingController _email;
 @override
 void initState() {
  super.initState();
  _name = TextEditingController(text: widget.employee?.name ?? '');
```

FLUTTER MINI ASSIGNMENT

```
  _age = TextEditingController(text: widget.employee?.age.toString() ?? '');
  _department = TextEditingController(
    text: widget.employee?.department ?? '',
  );
  _salary = TextEditingController(
    text: widget.employee?.salary.toString() ?? '',
  );
  _email = TextEditingController(text: widget.employee?.email ?? '');
 }
 Future<void> _submit() async {                                    //validate forms and all necessary
steps
  if (!_formKey.currentState!.validate()) return;
  final employee = Employee(
    id: widget.employee?.id,
    name: _name.text,
    age: int.parse(_age.text),
    department: _department.text,
    salary: double.parse(_salary.text),
    email: _email.text,
  );
  await widget.onSave(employee);
  if (mounted) Navigator.pop(context);
 }
 @override
 Widget build(BuildContext context) {
  final isEditing = widget.employee != null;
  return Padding(
    padding: EdgeInsets.all(16),
    child: Form(
      key: _formKey,
      child: Column(
        mainAxisSize: MainAxisSize.min,
        children: [
```

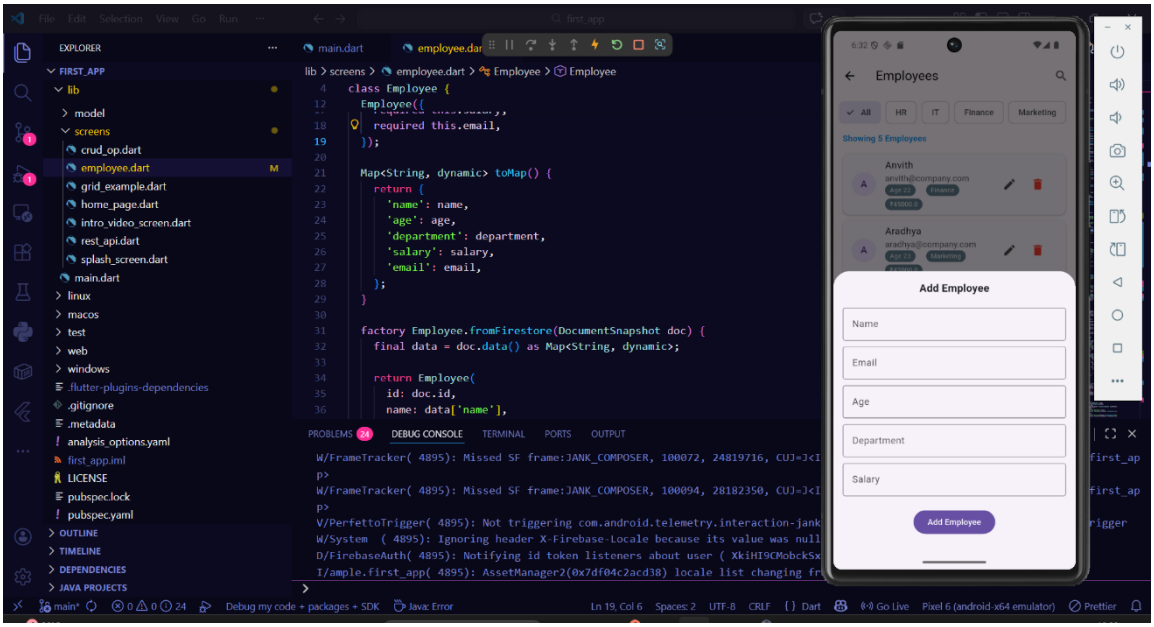
```
      Text(
        isEditing ? "Edit Employee" : "Add Employee",
        style: TextStyle(fontSize: 18, fontWeight: FontWeight.bold),
      ),
      SizedBox(height: 20),
      _Field(controller: _name, label: "Name"),
      SizedBox(height: 10),
      _Field(controller: _email, label: "Email"),
      SizedBox(height: 10),
      _Field(
        controller: _age,
        label: "Age",
        keyboardType: TextInputType.number,
      ),
      SizedBox(height: 10),

      _Field(controller: _department, label: "Department"),
      SizedBox(height: 10),
      _Field(
        controller: _salary,
        label: "Salary",
        keyboardType: TextInputType.number,
      ),
      SizedBox(height: 20),
      FilledButton(
        onPressed: _submit,
        child: Text(isEditing ? "Save Changes" : "Add Employee"),
      ),
      SizedBox(height: 40),
    ],
  ),
),
);
```

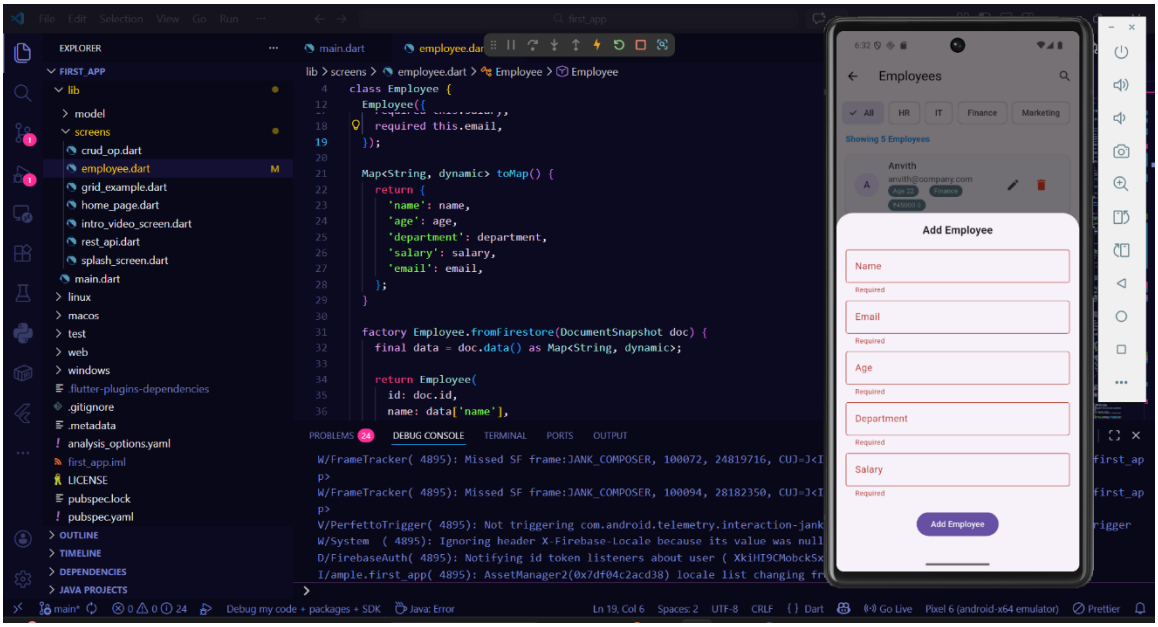
FLUTTER MINI ASSIGNMENT

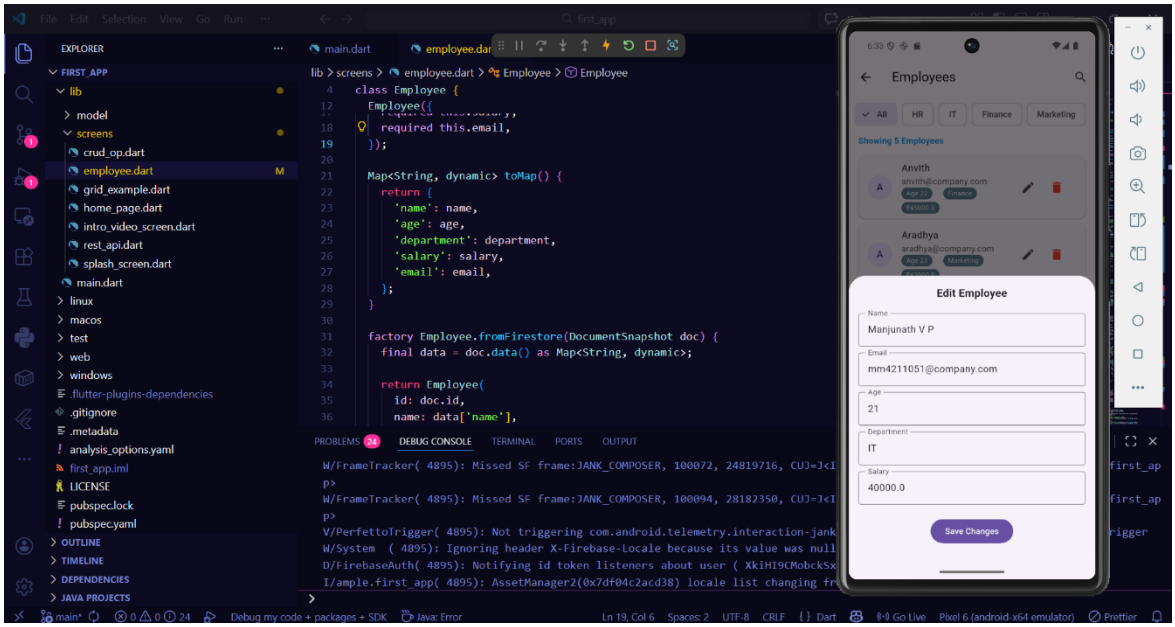
```
  }
}
class _Field extends StatelessWidget {
  final TextEditingController controller;
  final String label;
  final TextInputType? keyboardType;
  const _Field({
    required this.controller,
    required this.label,
    this.keyboardType,
  });
  @override
  Widget build(BuildContext context) {
    return TextFormField(
      controller: controller,
      keyboardType: keyboardType,
      validator: (value) => value == null || value.isEmpty ? "Required" : null,                    //validation
      decoration: InputDecoration(
        labelText: label,
        border: OutlineInputBorder(),
      ),
    );
  }
}
```

FLUTTER MINI ASSIGNMENT

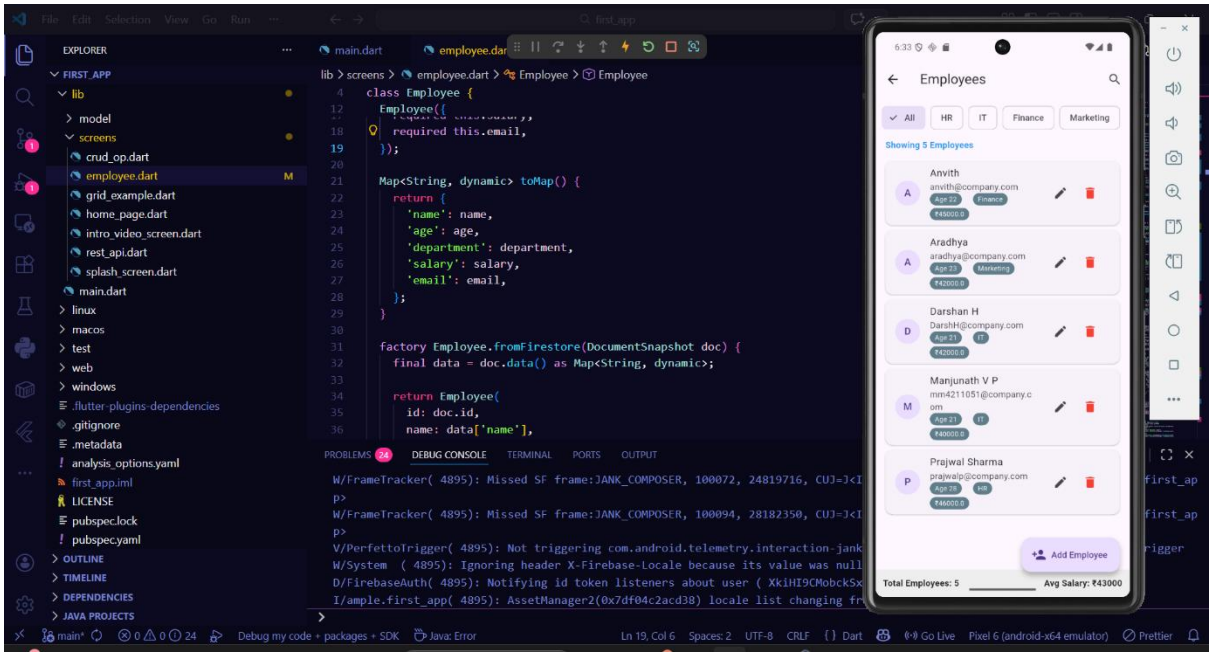
//Output

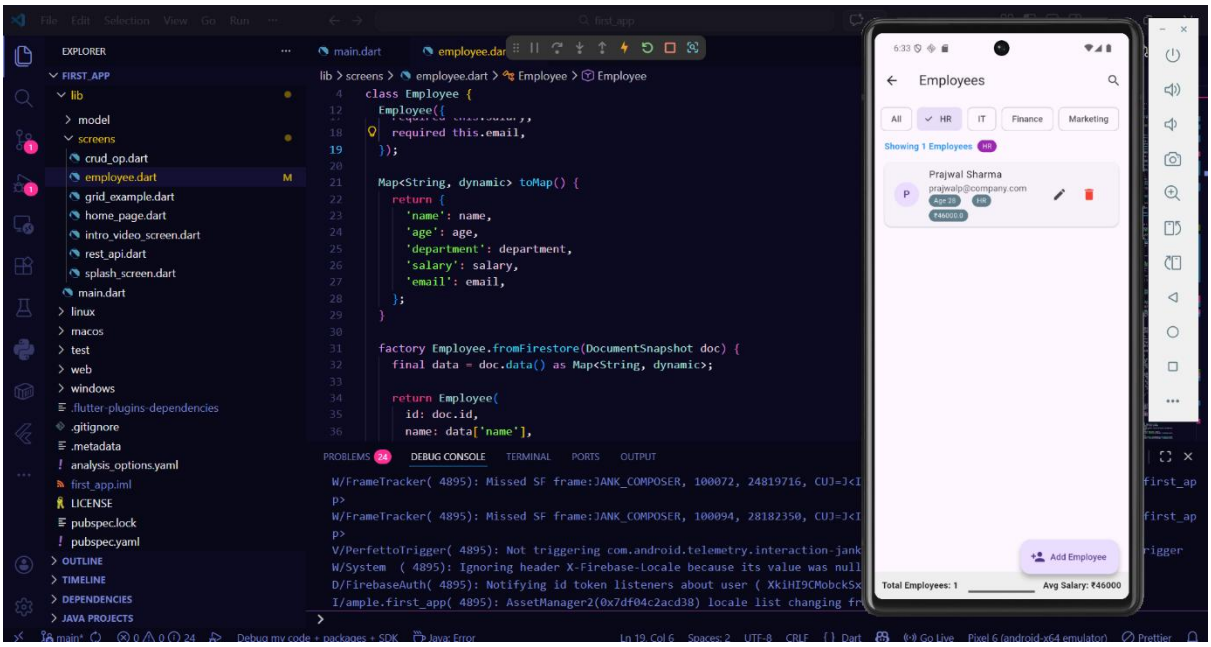


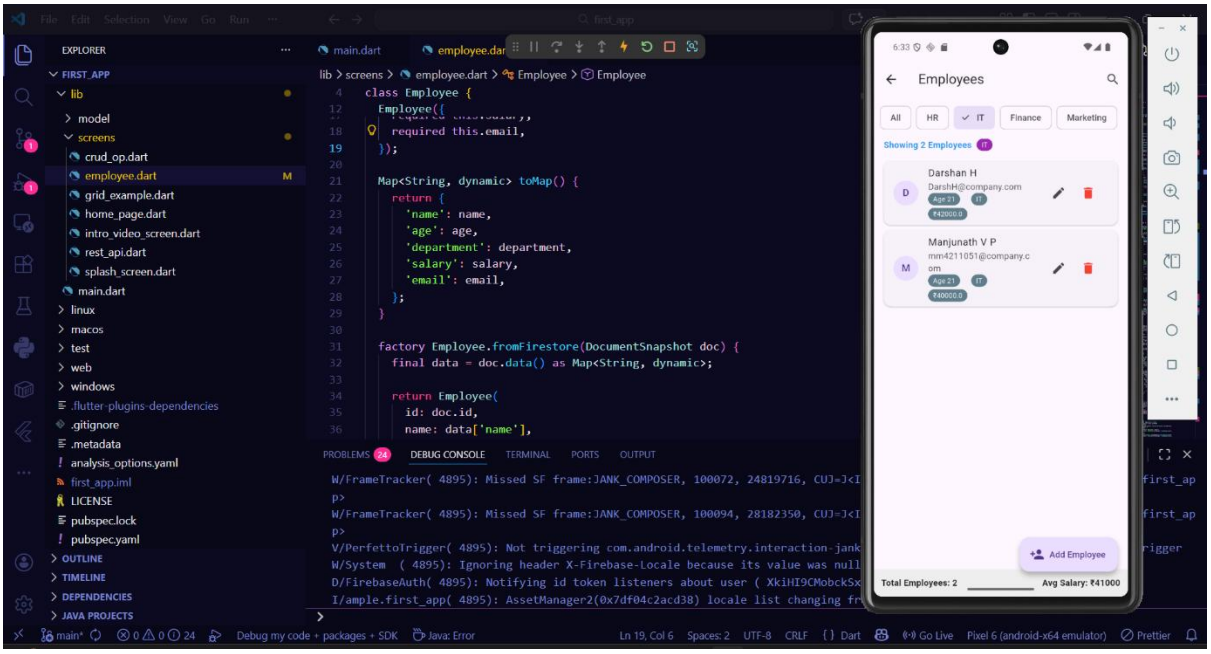




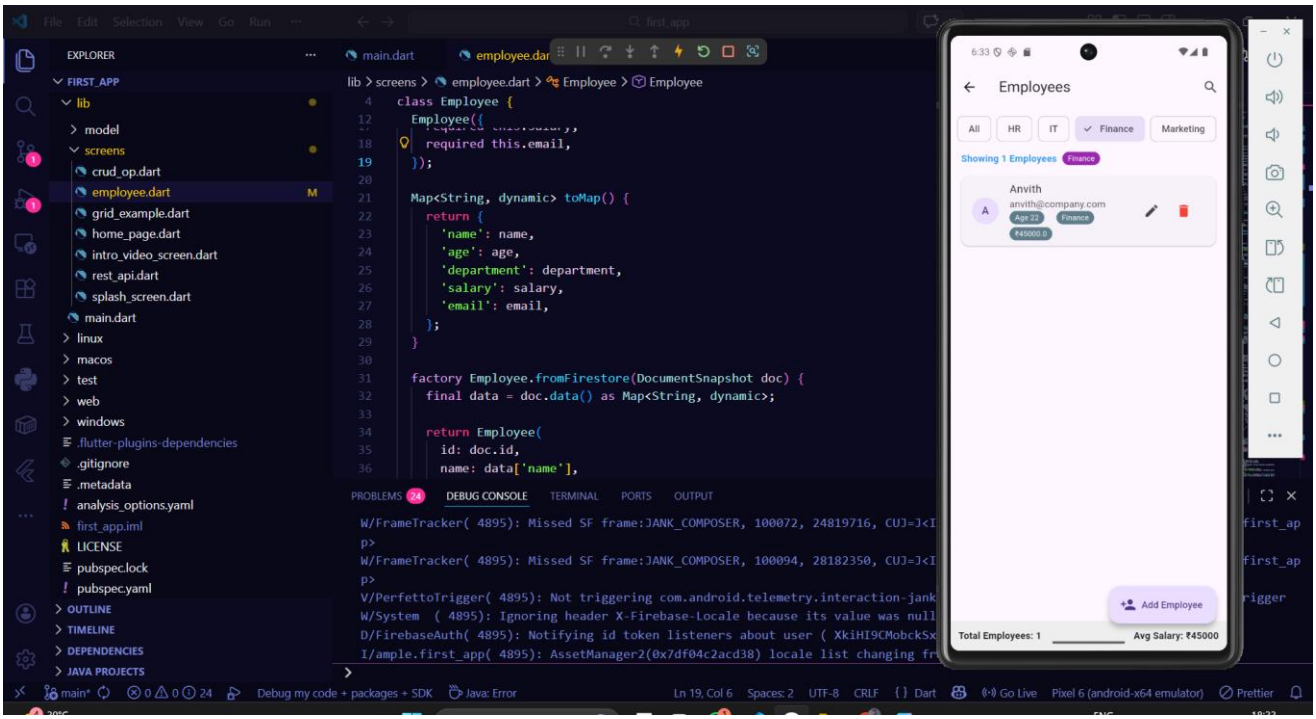
FLUTTER MINI ASSIGNMENT

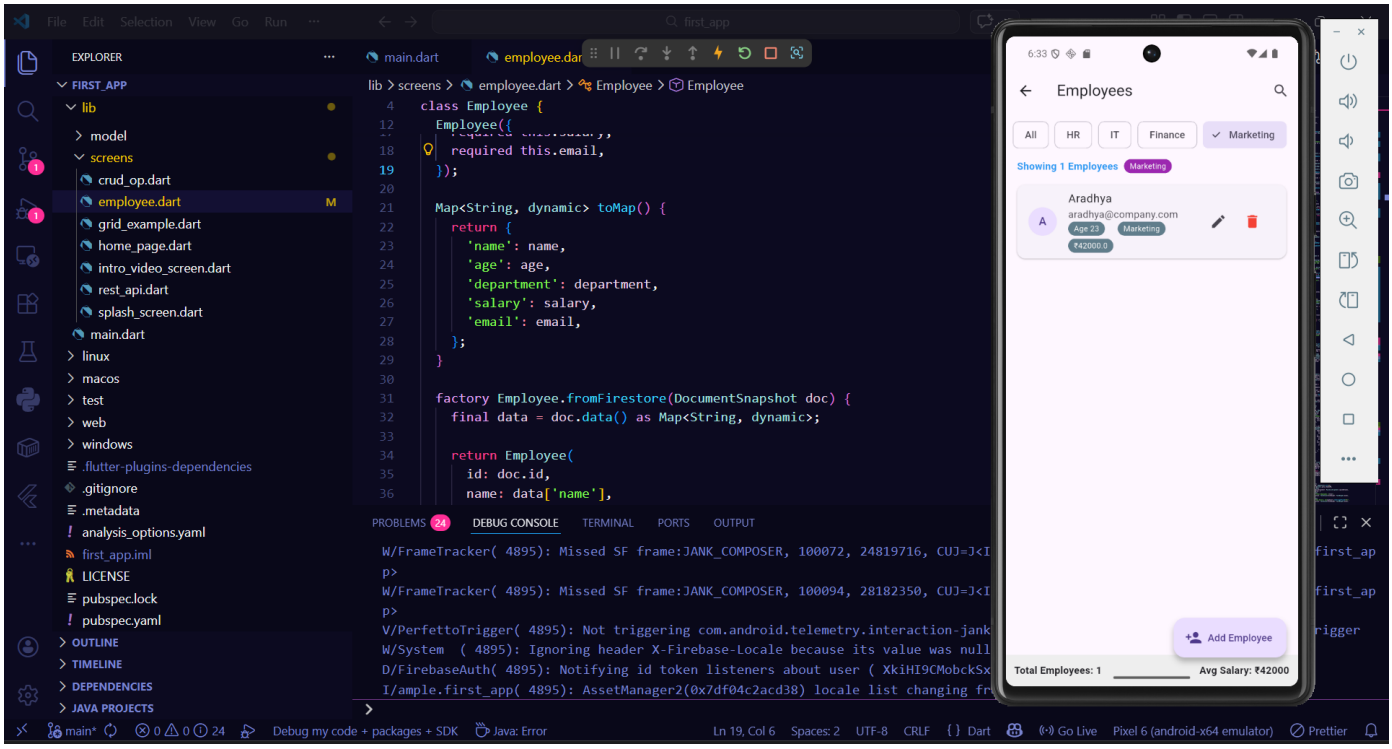
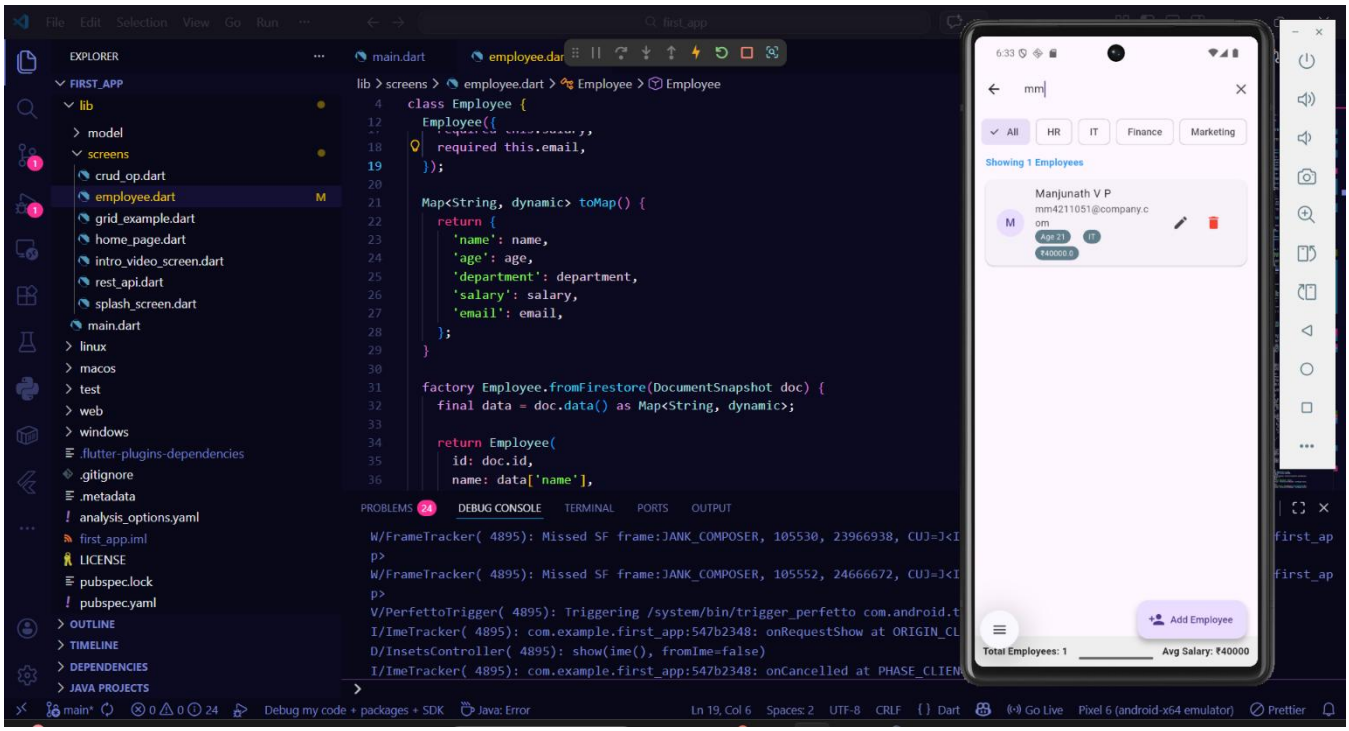
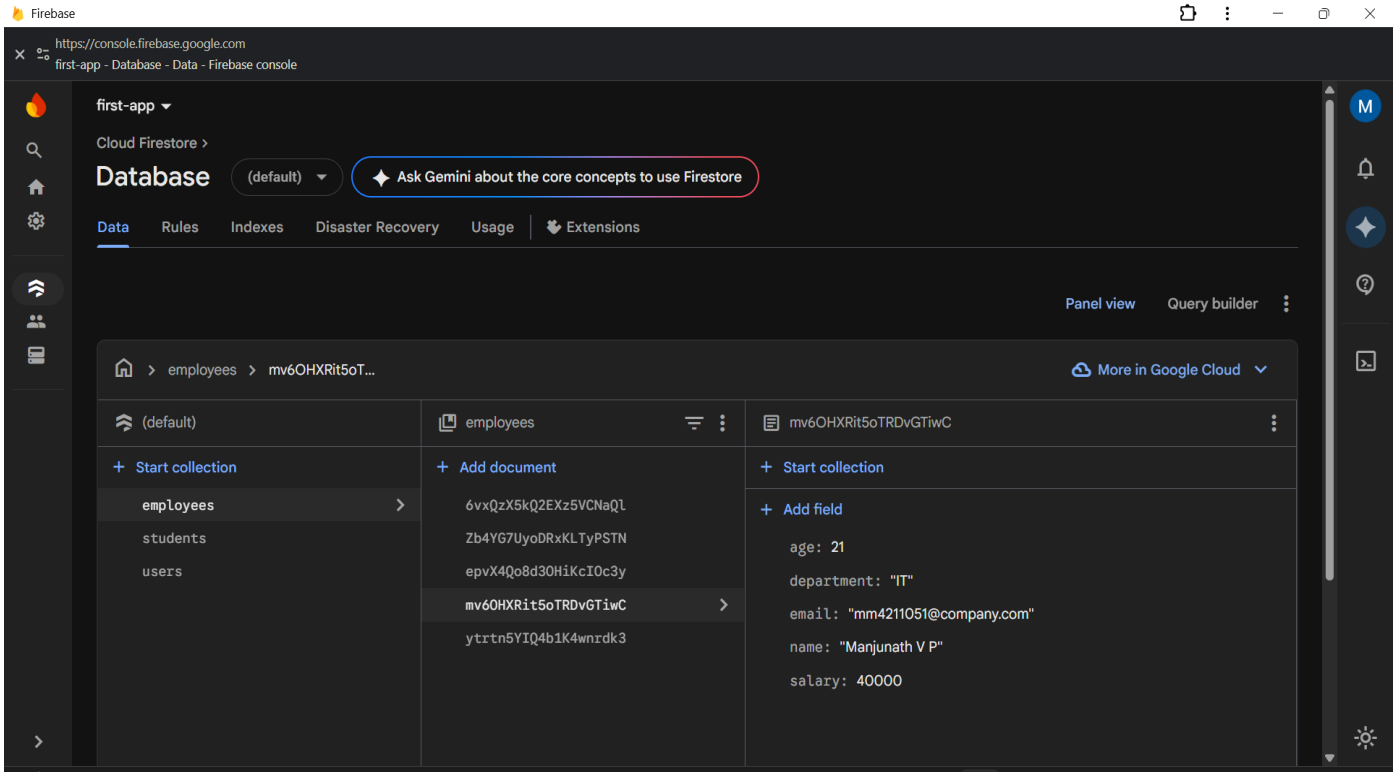






FLUTTER MINI ASSIGNMENT





FLUTTER MINI ASSIGNMENT